

Matthew Goodman

Technical Portfolio

Calibration / validation, semiconductor manufacturing, machine vision + automation, computer vision, compiler research, and numerical linear algebra.

The strongest common thread is practical: understand the physical system, measure it carefully, then build software or process controls that make the result more repeatable.

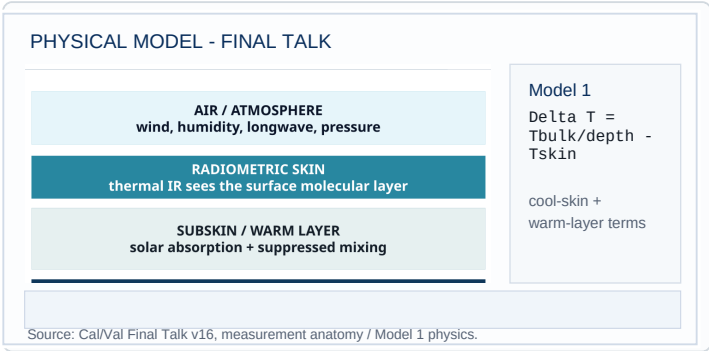
- | | | |
|----|------------------------------|---|
| 01 | JPL thermal Cal/Val | Physics-first bulk-to-skin temperature correction, large-scale validation, residual ML, and audit workflows. |
| 02 | Kulite laser-trim automation | OpenCV inspection, bounded zoom/focus optimization, N/E/S/W localization, stage offsets, and trim/verify control. |
| 03 | Kulite leadless process | CF7578 particle-size engineering, Ag/Pt/Pd conductive-glass fill, particle image analysis, and polished cross-section verification. |
| 04 | Baseball pitch tracking | Broadcast-frame detection, SORT tracking, interpolation, trajectory metrics, and pitch classification. |
| 05 | Stephen Edwards Lab | SSlang compiler research: typed-IR optimization, regression tests, lexical/syntactic analysis, and compiler-pass work. |
| 06 | PCA / eigenfaces | SVD, rank-k reconstruction, projection error, and low-dimensional representation of face images. |

JPL — bulk-to-skin temperature modeling

Physical comparator + ML benchmark
Large-scale timestamped validation

Problem

Translate in-situ bulk-water measurements and meteorological forcing into a physically defensible skin-temperature reference, then evaluate correction models against Level-2 station truth.



Selected Tahoe benchmark

0.402 °C

physics model RMSE

0.279 °C

XGBoost RMSE

1.18M

exact-row benchmark

Model labels and comparisons kept separate so "physics improvement" and "ML replay" are not conflated.

Distributed held-out block split — Salton Sea

A 5-minute block split distributes train/test coverage across the full record while preventing adjacent samples from leaking across the boundary.

Model	Held-out RMSE	Held-out rows
LSTM	0.23764 °C	960,951
XGBoost	0.25096 °C	960,951

What the project demonstrates

- Physics-first modeling with explicit correction terms and sign/unit checks.
- Multi-million-row data joins and station-level truth filtering.
- Chronological and distributed-block evaluation protocols.
- Cross-station monthly audits and anomaly flagging before model interpretation.
- Reproducible research packaging rather than one-off notebooks.

```
# Model 4: station bulk + locked MET/time -> James skin

features = [station_bulk, MET, time]
train, test = split_whole_5min_blocks(rows)
model.fit(train[features], train[Tskin_L2])

pred = model.predict(test[features])

report_metrics(pred, test[Tskin_L2])

same truth / rows / split / QC for comparisons
```

Public-facing portfolio excerpt: the emphasis is the modeling and validation method. Internal data paths, operational details, and non-public station files are not included.

Kulite - vision-guided laser trimming & stage localization

C#.NET · OpenCV · motion control

sanitized engineering reconstruction

The machine-vision work was not just image classification: it sat inside a motion-and-process loop. The system had to localize the target, get to a useful optical state quickly, correct stage pose, evaluate the cut, trim, and verify before stepping to the next location.

CLOSED-LOOP TRIMMING WORKFLOW

acquire → localize → zoom / focus → cut check → trim → verify

Inspection result feeds the next stage / laser action instead of ending at a screenshot.

CONSTRAINED ZOOM + FOCUS OPTIMIZATION

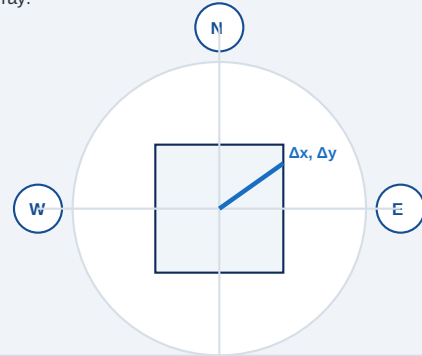
To zoom in faster, I treated the lens state as a bounded optimization problem rather than repeatedly sweeping the full range. The search maximized usable image sharpness while keeping required reference geometry in view.

```
maximize sharpness(z, f) - λ · travel(z, f)
subject to fiducials_visible = true
zmin ≤ z ≤ zmax, fmin ≤ f ≤ fmax
```

Warm-start near the previous good setting; refine only inside the feasible region.

NORTH / EAST / SOUTH / WEST LOCALIZATION

Cardinal edge searches gave a robust local center and offset before each stage step. The same logic can be applied repeatedly as the stage moves across the wafer / part array.



```
cx = (xE + xW)/2    cy = (yN + yS)/2
offset = (cx - xref, cy - yref)
step → re-localize → apply offset → inspect / trim
```

REPRESENTATIVE IMPLEMENTATION PATTERN

```
var edgeN = FindEdge(frame, North);
var edgeE = FindEdge(frame, East);
var edgeS = FindEdge(frame, South);
var edgeW = FindEdge(frame, West);

var center = Midpoint(edgeE, edgeW, edgeN, edgeS);
var offset = center - expectedCenter;
stage.Translate(-offset.X, -offset.Y);

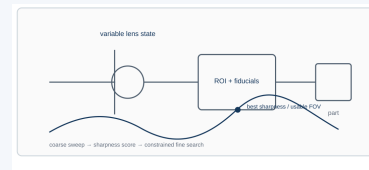
var optics = OptimizeBounded(
    zoomFocusState,
    score: SharpnessMinusTravel,
    constraint: FiducialsRemainVisible);

if (!CompleteCutDetected(camera.Grab()))
    laser.Execute(BuildTrimPath());

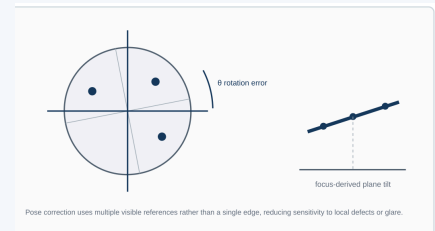
stage.StepToNextSite(); // repeat localization
```

OPTICAL + POSE SCHEMATICS

Optical optimization: zoom + focus



Wafer rotation + tilt compensation



Genericized reconstruction; dimensions and production setpoints omitted.

Portfolio reconstruction: exact device geometry, laser recipe, machine limits, thresholds, and production source code are intentionally omitted.

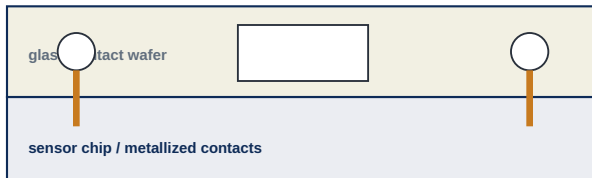
Kulite - leadless interconnect & particle analysis

glass-frit + Ag/Pt/Pd interconnect

CF7578 · Ag/Pt/Pd · image analysis

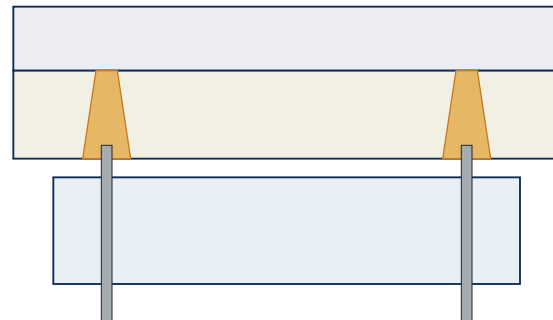
Leadless replaces fragile fine gold-wire interconnects with a fired metal-glass composite through the contact wafer to the header pins. CF7578 glass frit is dispersed through Heraeus Ag/Pt/Pd conductor paste; my work focused on material preparation, fill quality, particle distribution, and polished cross-sections.

PUBLIC LEADLESS DESIGN - PUBLISHED KULITE ARCHITECTURE



redrawn from Kulite Fig. 2-18 / Fig. 5

Left: sensor chip + glass contact wafer before / after bonding.



Right: fired CF7578 + Ag/Pt/Pd paste composite fills contact holes to header pins.

sensor / contact glass / fired metal-glass composite / header pins

redrawn from Kulite Fig. 2-20 / Fig. 7

MATERIAL / PROCESS WORK

Started with CF7578 glass frit. I sent the glass for ultra-fine milling, reducing average particle size from about 14 μm to about 4 μm , then dispersed the milled glass through Heraeus Ag/Pt/Pd conductor paste. The leadless fill is a metal-rich paste matrix containing glass particles - not glass alone.

~14 $\rightarrow$ ~4 μm

average particle-size reduction before blending / fill

LEADLESS FILL SEQUENCE

mill glass

blend Ag/Pt/Pd

fill narrow / deep aperture

align to e-base pin

vacuum / pressure fire

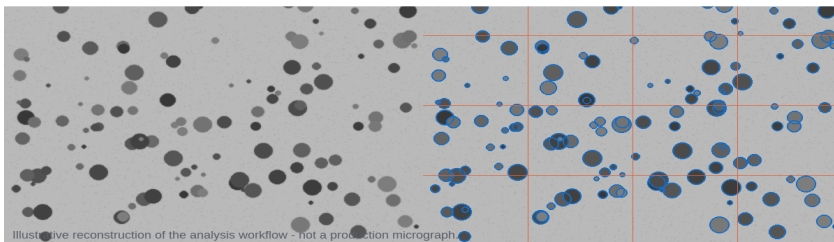
pot + polish

The filled CF7578 + Ag/Pt/Pd paste is processed under vacuum / pressure and fired. The fired composite forms the electrical path; glass particles remain distributed through the light-gray metallic paste matrix.

PARTICLE CONCENTRATION + DISTRIBUTION IMAGE ANALYSIS

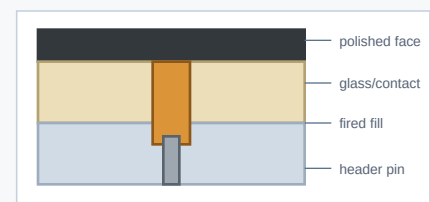
light-gray Ag/Pt/Pd paste + dispersed CF7578 glass

detected glass particles + spatial density



I performed image analysis specifically on the glass particles dispersed throughout the light-gray Ag/Pt/Pd paste matrix. Thresholding / connected components quantified local particle concentration and spatial distribution, making clusters, sparse regions, and oversized particles visible instead of collapsing the material into one bulk particle-size number.

POT + POLISH VERIFICATION



Cross-section inspection checks:

- complete fill / penetration
- voids or cracks
- wetting at the header-pin interface
- intended electrical contact after fire

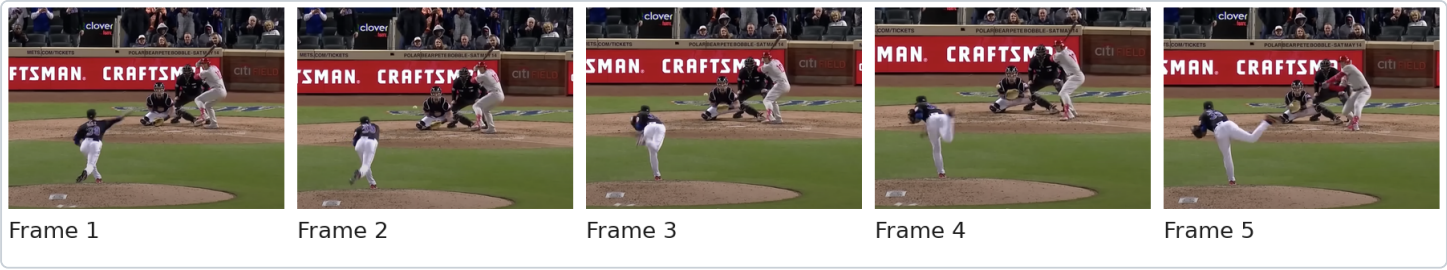
Distinct verification view - not the architecture diagram above.

Public design sources: Kulite Pressure Transducer Handbook, Sec. 2.8.6 (Figs. 2-18 to 2-20); Improved Ruggedized SOI Transducers Above 600°C (Figs. 4-7).

The architecture drawings above are redrawn from those public figures; the particle-analysis graphic is a sanitized reconstruction, not a production micrograph.

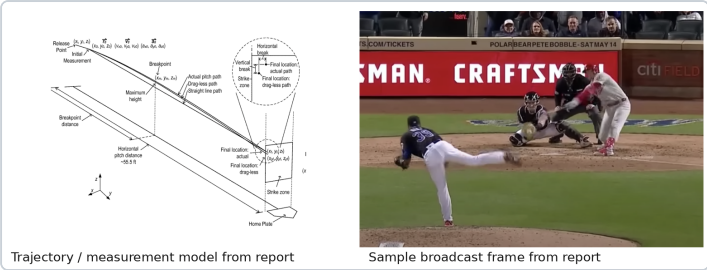
Baseball pitch tracking

from broadcast video
OpenCV · SORT · trajectory



Actual frames embedded in the submitted report, shown as a pitch sequence rather than a stock illustration.

Tracking pipeline



The report used behind-the-pitcher broadcast footage, frame sampling, TensorFlow object detection, SORT tracking, and interpolation for lost detections. Coordinates were then converted into motion statistics and rule-based pitch labels.

19 / 20

ball / strike classification

17-19 / 20

pitch-type binary tests

20 videos

reported evaluation set

Actual implementation excerpt

```
vid = cv2.VideoCapture(video_path)
tracker = Sort(max_age=8,
               min_hits=tracker_min_hits,
               iou_threshold=0.1)

return_value, frame = vid.read()
frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

detections, ballpos = detect(
    infer, frame, input_size, iou,
    score_threshold, detected_balls)

if len(detections) > 0:
    trackings = tracker.update(np.array(detections))
else:
    trackings = tracker.update()

for t in trackings:
    centerX = int((t[0] + t[2]) / 2)
    centerY = int((t[1] + t[3]) / 2)
    tracked_balls.append([centerX, centerY, color])

fill_lost_tracking(pitch_frames)
```

Example coordinate output

(829,447) → (838,430) → (850,413) → ... → (939,504) → (943,532)

The report then computed vertical/horizontal movement, approximate speed, strike/ball, and pitch type from the tracked sequence.

Useful interview angle: this is a complete visual-computing system, not just a classifier: video ingestion → detection → tracking → missing-data handling → trajectory features → decision logic → overlay output.

Stephen Edwards Lab - SSlang compiler research

Columbia CS · Stephen A. Edwards

SSlang · Sparse Synchronous Model · draft PR #159

Research on SSlang, an under-development language and compiler for the Sparse Synchronous Model. My draft PR #159 targeted a compiler optimization around par expressions: if every argument is instantaneous, the pass can replace redundant parallel structure with a simpler tuple-like IR form; if an argument may wait, block, or is not proven instantaneous, the par semantics must be preserved.

OPTIMIZATION / REGRESSION INTENT

```
let q = par 2 + 3    // instantaneous
          3 + 4    // instantaneous

// q may be rewritten as a tuple

let r = par add x y  // not assumed instantaneous
          add y x

// r must remain a par

-- pass operates on typed IR after lowering
```

RELATED LANGUAGE ANALYSIS

Lexical analysis maps source characters into tokens such as identifiers, literals, operators, and reserved forms. Syntactic analysis then applies grammar and precedence to build structured expressions / an AST.

SEMANTIC / IR TRANSFORM

The optimization belongs later in the pipeline: inspect typed IR nodes, prove the instantaneous-argument condition, perform a semantics-preserving rewrite, and use negative regression cases to make sure timing behavior is not changed accidentally.

RESEARCH CONTRIBUTION

The draft contribution adds the par optimization path, regression cases spanning all-instantaneous, mixed, and non-instantaneous arguments, and an IR dump mode to inspect the compiler state immediately after the pass.

DRAFT / OPEN PR

Research framing - not presented as a merged or production compiler feature.

Public references: cs.columbia.edu/~sedwards | github.com/ssm-lang/sslang/pull/159

PCA / eigenfaces via SVD

low-rank approximation · projection error

Implemented eigenfaces from a centered face-image matrix using SVD, then projected new faces and non-face images from the original lab into the learned face subspace. Projection residual quantified how well each image matched the learned facial-structure basis.

Low-rank reconstruction



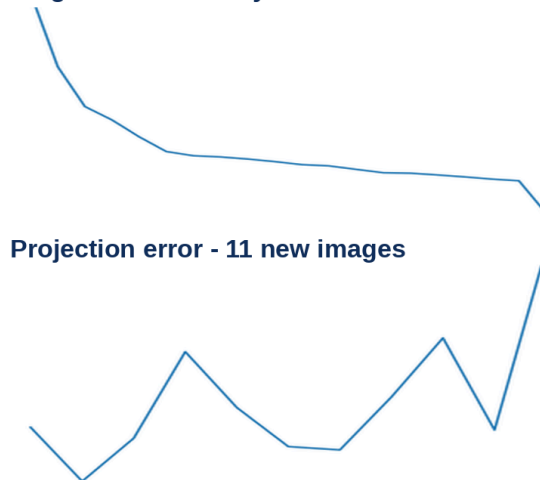
Mean face

Rank 5

Rank 10

Selected notebook outputs shown without overlap

Singular-value decay



Projection error - 11 new images

Notebook outputs: selected low-rank reconstructions, singular-value decay, and projection-error plot.

Core computation

```
mean_image = np.mean(faces_array, axis=0)
U, S, VT = svd(faces_array - mean_image)

sigma = np.diag(S)

def rank(k):
    Xk = U[:, :k] @ sigma[:k, :k] @ VT[:k, :]
    return Xk + mean_image

for k in range(1, 21):
    residual = faces_array - rank(k)
    error.append(np.linalg.norm(residual, 'fro'))
```

Linear-algebra interpretation

- Right singular vectors of the centered data matrix form the principal-component directions.
- Truncating to the first k singular directions gives the best rank- k approximation under squared error.
- The falling reconstruction error visualizes how much structure is captured as k increases.
- Projection residual provides a simple anomaly/class-membership signal: images poorly represented by the face subspace produce larger error.

Why it belongs in the portfolio: it shows the same mathematical pattern that recurs in calibration, signal processing, robotics, and finance — choose a basis that captures the dominant structure, then reason about what remains in the residual.

Actual lab test: household object projected into the face subspace



Actual lab household object
(unclassified image 0)



Actual eigenface projection
from the lab output



Projection-residual test

$$|| x - P_{\text{face}} x ||$$

smaller residual = more face-like